

Assignment 6- CP363

Final documentation

Moosa Ahmed, Muhammad Saim Bin Asif

169046796, 169075698

Group # 26

Equipment Test Center

Dr. Haytham Qushtom

Due: November 16th, 2025

Introduction

This project designs a database system for managing laboratory equipment testing and client bookings. The system organizes information about clients, technicians, laboratories, equipment, tests, test sessions, invoices, reports, and maintenance records. The goal is to ensure efficient scheduling, accurate record-keeping, and proper tracking of testing activities, while avoiding redundancy and maintaining data consistency. This document presents the key system entities, the main structural features of the design, the final 3NF relational schema, and the complete normalization process used to produce it.

Key Features

- **10 fully defined entities** with clear attributes, primary keys, foreign keys, and constraints.
- **Normalized to BCNF**, ensuring minimal redundancy and strong data integrity.
- **All business rules are enforced** through structured relationships, participation constraints, and schema design.
- **Support for analytical reporting** through well-organized tables, enabling optimized queries and view creation.

Business Rules

1. Clients must register before making equipment test bookings.
2. Each booking must specify at least one piece of equipment and is assigned to exactly one lab.
3. Every booking generates exactly one invoice.
4. A technician must supervise every test session.
5. Each test session involves one predefined test and may use one or more equipment items.
6. Each test session produces exactly one report.
7. Equipment must undergo regular maintenance, and each maintenance record is tied to one equipment (weak entity).
8. A lab can host multiple bookings, but equipment can only be used in one booking at the same time.
9. A client can have multiple invoices, but each invoice belongs to exactly one booking.

Final Relational Schema (3NF)

Below is the final normalized schema with primary keys (PK) and foreign keys (FK).

Equipment

- Equipment_ID (PK)
- Name
- Model_Number
- Manufacturer
- Status

Client

- Client_ID (PK)
- First_Name
- Last_Name
- Contact_Email (UNIQUE)
- Phone_Number (UNIQUE)
- Organization

Technician

- Technician_ID (PK)
- First_Name
- Last_Name
- Certification
- Contact_Email (UNIQUE)
- Phone_Number (UNIQUE)

Test

- Test_ID (PK)
- Test_Name
- Description

Test_Session

- Session_ID (PK)
- Test_Date
- Result
- Duration
- Technician_ID (FK)
- Test_ID (FK)

Lab

- Lab_ID (PK)
- Lab_Name
- Location
- Capacity

Booking

- Booking_ID (PK)
- Booking_Date
- Client_ID (FK)
- Lab_ID (FK)
- Start_Time
- End_Time
- Status

Report

- Report_ID (PK)
- Report_Date
- Findings
- Approved_By (FK → Technician)
- Session_ID (FK, UNIQUE)

Invoice (*Weak Entity*)

- Invoice_ID (PK)
- Invoice_Date
- Booking_ID (FK, UNIQUE)
- Amount
- Payment_Status

Maintenance_Record (*Weak Entity*)

- Record_ID (PK)
- Equipment_ID (FK)
- Maintenance_Date
- Technician_ID (FK)
- Notes

Booking_Equipment (*M:N Table*)

- Booking_ID (FK)
 - Equipment_ID (FK)
- PK = (Booking_ID, Equipment_ID)

TestSession_Equipment (*M:N Table*)

- Session_ID (FK)
 - Equipment_ID (FK)
- PK = (Session_ID, Equipment_ID)

Normalization Process

- Below is the complete normalization process up to Third Normal Form (3NF).

First Normal Form (1NF)

Requirements:

- No repeating groups
- All fields are atomic
- Each table has a primary key

All attributes are atomic (no multivalued fields).

M:N relationships handled with linking tables → Booking_Equipment and TestSession_Equipment.

- All tables satisfy 1NF

Second Normal Form (2NF)

Requirements:

- In 1NF
- No partial dependencies (non-key attributes depending on *part* of a composite key)

Most tables have simple primary keys → automatically 2NF. Linking tables (Booking_Equipment & TestSession_Equipment) have composite keys but **no non-key attributes**, so no partial dependencies.

- All tables satisfy 2NF

Third Normal Form (3NF)

Requirements:

- In 2NF
- No transitive dependencies
- Non-key attributes must depend only on the PK

Checks:

- Entity tables have no attributes depending on other non-key attributes.
- Email/phone fields are UNIQUE → alternate candidate keys, not transitive dependencies.
1:1 tables (Report, Invoice) do not create transitive problems.
- Weak entities depend only on their PKs.
- All tables satisfy 3NF

Relation algebra (RA) for queries

Query 1: List all bookings with client names

RA Expression:

π Booking_ID, First_Name, Last_Name, Booking_Date, Status
(Booking $\bowtie$ {Booking.Client_ID = Client.Client_ID} Client)

Query 2: Equipment currently in use

RA Expression:

π Equipment_ID, Name, Status
(σ {Status = 'In Use'} (Equipment))

Query 3: Clients with confirmed bookings

RA Expression:

π First_Name, Last_Name
(Client $\bowtie$ {Client.Client_ID = Booking.Client_ID} σ {Status = 'Confirmed'}(Booking))

Query 4: Tests that never failed

RA Expression:

π Test_Name(Test) $- \pi$ Test_Name (Test $\bowtie$ {Test.Test_ID = Test_Session.Test_ID} σ {Result = 'Fail'}(Test_Session))

Query 5: Clients with their latest booking date

RA Expression:

Latest $\leftarrow \gamma$ Client_ID ; MAX(Booking_Date) $\rightarrow$ Latest_Booking (Booking)

π First_Name, Last_Name, Latest_Booking
(Client $\bowtie$ {Client.Client_ID = Latest.Client_ID} Latest)

Query 6: Client booking ranking (using count + rank logic)

Counts $\leftarrow \gamma$ Client_ID ; COUNT(Booking_ID) $\rightarrow$ Total_Bookings (Booking)
RankedClients $\leftarrow \gamma$ Client_ID, Total_Bookings ; RANK() $\rightarrow$ Booking_Rank (Counts)

π Client_ID, First_Name, Last_Name, Total_Bookings, Booking_Rank
(Client $\bowtie$ {Client.Client_ID = RankedClients.Client_ID} RankedClients)

Query 7: Technician session numbering

Sessions $\leftarrow \gamma$ Technician_ID, Session_ID, Test_Date ; ROW_NUMBER() $\rightarrow$ Session_Number
(Test_Session)

π Technician_ID, Session_ID, Test_Date, Session_Number (Sessions)

Query 8: Booking groups using NTILE(2)

$\text{Groups} \leftarrow \gamma \text{ Booking_ID, Booking_Date ; NTILE(2)} \rightarrow \text{Booking_Group (Booking ORDER BY Booking_Date)}$

$\pi \text{ Booking_ID, Booking_Date, Booking_Group (Groups)}$

Query 9: Reports with approving technicians

RA Expression:

$\pi \text{ Report_ID, Findings, First_Name, Last_Name}$

$(\text{Report} \bowtie_{\text{Report.Approved_By} = \text{Technician.Technician_ID}} \text{Technician})$

Query 10: Invoices above average amount

$\text{Avg} \leftarrow \gamma \text{ ; AVG(Amount)} \rightarrow \text{AvgAmount_ (Invoice)}$

$\pi \text{ Booking_ID, Invoice_ID, Amount, Payment_Status}$

$(\sigma_{\text{Amount} > \text{AvgAmount}} (\text{Invoice} \times \text{Avg}))$

Python code (GUI)

```
10 """
11 -----
12 CP 363 Assignment 6 GUI
13 -----
14 Author: Moosa Ahmed
15 ID: W20603301
16 Email: W20603301@learning.laurieric.ca
17 __updated__ = Nov. 15, 2025
18 -----
19 """
20
21 import mysql.connector
22 import tkinter as tk
23 from tkinter import ttk, messagebox
24
25 # Database Connection
26 conn = mysql.connector.connect(
27     host="localhost",
28     user="root",
29     password="36990242@MySQL",
30     database="EquipmentTestCenterDB"
31 )
32 cursor = conn.cursor()
33
34 # Main Window
35 root = tk.Tk()
36 root.title("Equipment Test Center Management")
37 root.geometry("1000x600")
38 root.resizable(True, True)
39
40 # Main Frame
41 main_frame = tk.Frame(root)
42 main_frame.pack(fill="both", expand=True)
43
44
45 # Helper Functions
46 def create_treeview(parent, columns, headers):
47     """Creates a scrollable Treeview inside the parent frame for displaying tabular data."""
48     for widget in parent.winfo_children():
49         widget.destroy() # Clear previous content
50     tree_frame = tk.Frame(parent)
51     tree_frame.pack(fill="both", expand=True)
52     tree_scroll = tk.Scrollbar(tree_frame)
53     tree_scroll.pack(side=tk.RIGHT, fill=tk.Y)
54
55     # Treeview setup
56     tree = ttk.Treeview(tree_frame, columns=columns, show="headings", yscrollcommand=tree_scroll.set)
57     tree.pack(fill="both", expand=True)
58     tree_scroll.config(command=tree.yview)
59
60     for col, head in zip(columns, headers):
61         tree.heading(col, text=head)
62         tree.column(col, width=120) # Default column width
63     return tree
64
65 def clear_frame():
66     """Clears all widgets from the main frame for new content display."""
67     for widget in main_frame.winfo_children():
68         widget.destroy()
```

```

59
60 # Back Button
61 def back_button(return_func=None):
62     """
63     Adds a Back button.
64     If 'return_func' is provided, navigates there; otherwise returns to main menu.
65     """
66     if return_func is None:
67         return_func = main_menu
68     tk.Button(main_frame, text="Back", command=return_func, bg="grey", fg="white").pack(pady=10)
69
70 # Display Functions
71 def show_equipment():
72     """Displays all equipment in the database."""
73     clear_frame()
74     tk.Label(main_frame, text="Equipment Records", font=("Arial", 16, "bold")).pack(pady=10)
75     cursor.execute("SELECT * FROM Equipment;")
76     rows = cursor.fetchall()
77     columns = ("Equipment_ID", "Name", "Model Number", "Manufacturer", "Status")
78     headers = ["ID", "Name", "Model", "Manufacturer", "Status"]
79     tree = create_treeview(main_frame, columns, headers)
80     for row in rows:
81         tree.insert("", "end", values=row)
82     back_button()
83
84 def show_clients():
85     """Displays all clients in the database."""
86     clear_frame()
87     tk.Label(main_frame, text="Client Records", font=("Arial", 16, "bold")).pack(pady=10)
88     cursor.execute("SELECT * FROM Client;")
89     rows = cursor.fetchall()
90     columns = ("Client_ID", "First Name", "Last Name", "Email", "Phone", "Organization")
91     headers = ["ID", "First Name", "Last Name", "Email", "Phone", "Organization"]
92     tree = create_treeview(main_frame, columns, headers)
93     for row in rows:
94         tree.insert("", "end", values=row)
95     back_button()

```

```

96
97 def show_bookings():
98     """Displays all bookings with client and lab info."""
99     clear_frame()
100    tk.Label(main_frame, text="Booking Records", font=("Arial", 16, "bold")).pack(pady=10)
101    cursor.execute("""
102        SELECT b.Booking_ID, c.First_Name, c.Last_Name, l.Lab_Name, b.Booking_Date, b.Start_Time, b.End_Time, b.Status
103        FROM Booking b
104        JOIN Client c ON b.Client_ID = c.Client_ID
105        JOIN Lab l ON b.Lab_ID = l.Lab_ID;
106    """)
107    rows = cursor.fetchall()
108    columns = ("Booking_ID", "Client First", "Client Last", "Lab", "Booking Date", "Start", "End", "Status")
109    headers = ["ID", "First Name", "Last Name", "Lab", "Date", "Start", "End", "Status"]
110    tree = create_treeview(main_frame, columns, headers)
111    for row in rows:
112        tree.insert("", "end", values=row)
113    back_button()
114
115 def show_reports():
116     """Displays all reports with technician approval and test session info."""
117     clear_frame()
118     tk.Label(main_frame, text="Reports", font=("Arial", 16, "bold")).pack(pady=10)
119     cursor.execute("""
120         SELECT r.Report_ID, r.Findings, t.First_Name, t.Last_Name, ts.Test_Date
121         FROM Report r
122         JOIN Technician t ON r.Approved_By = t.Technician_ID
123         JOIN Test_Session ts ON r.Session_ID = ts.Session_ID;
124     """)
125     rows = cursor.fetchall()
126     columns = ("Report_ID", "Findings", "Tech First", "Tech Last", "Test Date")
127     headers = ["ID", "Findings", "Technician First", "Technician Last", "Test Date"]
128     tree = create_treeview(main_frame, columns, headers)
129     for row in rows:
130         tree.insert("", "end", values=row)
131
132     # Advanced report: Total revenue from invoices
133     tk.Button(main_frame, text="View Total Revenue", command=show_total_revenue, bg="#f44336", fg="white").pack(pady=10)
134     back_button()

```

```

135
136 def show_total_revenue():
137     """Calculates and displays total revenue from all invoices. Handles NULL by defaulting to 0."""
138     cursor.execute("SELECT SUM(Amount) FROM Invoice;")
139     total = cursor.fetchone()[0] or 0 # NULL safe
140     messagebox.showinfo("Total Revenue", f"Total Revenue: ${total:.2f}")
141
142 # Advanced Queries
143 def show_advanced_queries():
144     """Shows the advanced queries menu."""
145     clear_frame()
146     tk.Label(main_frame, text="Advanced Queries", font=("Arial", 16, "bold")).pack(pady=10)
147
148     # Special case: Advanced reports
149     tk.Button(main_frame, text="Tests Never Failed", command=show_tests_never_failed, bg="#3F51B5", fg="white").pack(pady=5)
150     tk.Button(main_frame, text="Client Booking Ranking", command=show_client_ranking, bg="#795548", fg="white").pack(pady=5)
151     back_button() # Back to main menu
152
153 def show_tests_never_failed():
154     """
155     Advanced query: List tests that never failed.
156     Uses a subquery in WHERE clause to exclude tests with any 'Fail' result.
157     """
158     clear_frame()
159     tk.Label(main_frame, text="Tests Never Failed", font=("Arial", 16, "bold")).pack(pady=10)
160     cursor.execute("""
161         SELECT Test_Name
162         FROM Test
163         WHERE Test_ID NOT IN (
164             SELECT Test_ID
165             FROM Test_Session
166             WHERE Result = 'Fail'
167         );
168     """)
169     rows = cursor.fetchall()
170     columns = ("Test_Name",)
171     headers = ["Test Name"]
172     tree = create_treeview(main_frame, columns, headers)
173     for row in rows:
174         tree.insert("", "end", values=row)

```

```

175
176 # Special case: Back returns to Advanced Queries menu
177 back_button(return_func=show_advanced_queries)
178
179 def show_client_ranking():
180     """
181     Advanced query: Rank clients by number of bookings.
182     Demonstrates aggregation and ordering for reporting purposes.
183     """
184     clear_frame()
185     tk.Label(main_frame, text="Client Booking Ranking", font=("Arial", 16, "bold")).pack(pady=10)
186     cursor.execute("""
187         SELECT c.First_Name, c.Last_Name, COUNT(b.Booking_ID) AS Total_Bookings
188         FROM Client c
189         LEFT JOIN Booking b ON c.Client_ID = b.Client_ID
190         GROUP BY c.Client_ID
191         ORDER BY Total_Bookings DESC;
192     """)
193     rows = cursor.fetchall()
194     columns = ("First_Name", "Last_Name", "Total_Bookings")
195     headers = ["First Name", "Last Name", "Bookings"]
196     tree = create_treeview(main_frame, columns, headers)
197     for row in rows:
198         tree.insert("", "end", values=row)
199
200 # Back returns to Advanced Queries menu
201 back_button(return_func=show_advanced_queries)
202
203 # Main Menu
204 def main_menu():
205     """Displays the main menu with all options."""
206     clear_frame()
207     tk.Label(main_frame, text="Equipment Test Center", font=("Arial", 20, "bold")).pack(pady=20)
208     tk.Button(main_frame, text="View Equipment", width=25, command=show_equipment, bg="#4CAF50", fg="white").pack(pady=5)
209     tk.Button(main_frame, text="View Clients", width=25, command=show_clients, bg="#2196F3", fg="white").pack(pady=5)
210     tk.Button(main_frame, text="View Bookings", width=25, command=show_bookings, bg="#FF9800", fg="white").pack(pady=5)
211     tk.Button(main_frame, text="View Reports", width=25, command=show_reports, bg="#9C27B0", fg="white").pack(pady=5)
212     tk.Button(main_frame, text="Advanced Queries", width=25, command=show_advanced_queries, bg="#795548", fg="white").pack(pady=5)
213     tk.Button(main_frame, text="Exit", width=25, command=root.quit, bg="grey", fg="white").pack(pady=20)
214
215 # Start
216 main_menu()
217 root.mainloop()

```

Conclusion

Successes:

- Fully normalized database design up to **BCNF**, ensuring minimal redundancy and strong data integrity.
- All business rules successfully enforced through entity constraints, participation rules, and relationship structure.
- Queries, including joins, subqueries, and analytical operations were optimized for efficient data retrieval.

Challenges & Solutions:

- Managing weak entities (Invoice, Maintenance_Record) required careful key selection, Resolved by enforcing unique FK constraints and strong parent-child dependencies
- Handling M:N relationships for equipment usage and test sessions added complexity , Solved using junction tables (Booking_Equipment, TestSession_Equipment).
- Ensuring no equipment is double-booked, addressed through schema structure and can be reinforced with application-level validation.

Future Improvements:

- Implement triggers to automate maintenance reminders and ensure timely servicing of equipment.
- Add advanced analytical dashboards for lab utilization, technician workload, and equipment performance trends.
- Introduce stored procedures for booking validation, conflict checking, and automated report generation

Final tables (reference)

Equipment table

```
CREATE TABLE IF NOT EXISTS Equipment (  
    Equipment_ID INT AUTO_INCREMENT PRIMARY KEY,  
    Name VARCHAR(100) NOT NULL,  
    Model_Number VARCHAR(50) NOT NULL,  
    Manufacturer VARCHAR(100) NOT NULL,  
    Status VARCHAR(20) NOT NULL DEFAULT 'Available'  
    CHECK (Status IN ('Available','In Use','Under Maintenance'))  
);
```

Client table

```
CREATE TABLE IF NOT EXISTS Client (  
    Client_ID INT AUTO_INCREMENT PRIMARY KEY,  
    First_Name VARCHAR(50) NOT NULL,  
    Last_Name VARCHAR(50) NOT NULL,  
    Contact_Email VARCHAR(100) NOT NULL UNIQUE,  
    Phone_Number VARCHAR(20) NOT NULL UNIQUE,  
    Organization VARCHAR(100) NOT NULL  
);
```

Technician table

```
CREATE TABLE IF NOT EXISTS Technician (  
    Technician_ID INT AUTO_INCREMENT PRIMARY KEY,  
    First_Name VARCHAR(50) NOT NULL,  
    Last_Name VARCHAR(50) NOT NULL,  
    Certification VARCHAR(100) NOT NULL,  
    Contact_Email VARCHAR(100) NOT NULL UNIQUE,  
    Phone_Number VARCHAR(20) NOT NULL UNIQUE  
);
```

Test table

```
CREATE TABLE IF NOT EXISTS Test (  
    Test_ID INT AUTO_INCREMENT PRIMARY KEY,  
    Test_Name VARCHAR(100) NOT NULL,  
    Description VARCHAR(255) NOT NULL  
);
```

Lab table

```
CREATE TABLE IF NOT EXISTS Lab (  
    Lab_ID INT AUTO_INCREMENT PRIMARY KEY,  
    Lab_Name VARCHAR(100) NOT NULL,  
    Location VARCHAR(100) NOT NULL,  
    Capacity INT NOT NULL CHECK (Capacity > 0)  
);
```

Booking table (Client books Lab)

```
CREATE TABLE IF NOT EXISTS Booking (  
    Booking_ID INT AUTO_INCREMENT PRIMARY KEY,  
    Booking_Date DATE NOT NULL,  
    Client_ID INT NOT NULL,  
    Lab_ID INT NOT NULL,  
    Start_Time TIME NOT NULL,  
    End_Time TIME NOT NULL,  
    Status VARCHAR(20) NOT NULL DEFAULT 'Pending'  
    CHECK (Status IN ('Pending','Confirmed','Completed','Cancelled')),  
    FOREIGN KEY (Client_ID) REFERENCES Client(Client_ID),  
    FOREIGN KEY (Lab_ID) REFERENCES Lab(Lab_ID)  
);
```

Test_Session table (A session of a test conducted by a technician)

```
CREATE TABLE IF NOT EXISTS Test_Session (  
    Session_ID INT AUTO_INCREMENT PRIMARY KEY,  
    Test_Date DATE NOT NULL,  
    Result VARCHAR(100) NOT NULL,  
    Duration TIME NOT NULL,  
    Technician_ID INT NOT NULL,  
    Test_ID INT NOT NULL,  
    FOREIGN KEY (Technician_ID) REFERENCES Technician(Technician_ID),  
    FOREIGN KEY (Test_ID) REFERENCES Test(Test_ID)  
);
```

Report table (Test session results approved by a technician)

```
CREATE TABLE IF NOT EXISTS Report (  
    Report_ID INT AUTO_INCREMENT PRIMARY KEY,  
    Report_Date DATE NOT NULL,  
    Findings VARCHAR(255) NOT NULL,  
    Approved_By INT NOT NULL,  
    Session_ID INT NOT NULL UNIQUE,  
    FOREIGN KEY (Approved_By) REFERENCES Technician(Technician_ID),  
    FOREIGN KEY (Session_ID) REFERENCES Test_Session(Session_ID)  
);
```

Invoice table - FIXED: Separate primary key for auto_increment

```
CREATE TABLE IF NOT EXISTS Invoice (  
    Booking_ID INT NOT NULL,  
    Invoice_Number INT NOT NULL,  
    Invoice_Date DATE NOT NULL,  
    Amount DECIMAL(10,2) NOT NULL CHECK (Amount >= 0),  
    Payment_Status ENUM('Paid','Unpaid') NOT NULL,  
    FOREIGN KEY (Booking_ID) REFERENCES Booking(Booking_ID),  
    PRIMARY KEY (Booking_ID, Invoice_Number)  
);
```

Maintenance Record table: Separate primary key for auto_increment

```
CREATE TABLE IF NOT EXISTS Maintenance_Record (  
    Equipment_ID INT NOT NULL,  
    Record_Number INT NOT NULL,  
    Maintenance_Date DATE NOT NULL,  
    Technician_ID INT NOT NULL,  
    Notes VARCHAR(255) NOT NULL,  
    FOREIGN KEY (Equipment_ID) REFERENCES Equipment(Equipment_ID),  
    FOREIGN KEY (Technician_ID) REFERENCES Technician(Technician_ID),  
    PRIMARY KEY (Equipment_ID, Record_Number)  
);
```

Booking_Equipment junction table (many-to-many relationship)

```
CREATE TABLE IF NOT EXISTS Booking_Equipment (  
    Booking_ID INT NOT NULL,  
    Equipment_ID INT NOT NULL,  
    PRIMARY KEY (Booking_ID, Equipment_ID),  
    FOREIGN KEY (Booking_ID) REFERENCES Booking(Booking_ID),  
    FOREIGN KEY (Equipment_ID) REFERENCES Equipment(Equipment_ID)  
);
```

TestSession_Equipment junction table (many-to-many relationship)

```
CREATE TABLE IF NOT EXISTS TestSession_Equipment (  
    Session_ID INT NOT NULL,  
    Equipment_ID INT NOT NULL,  
    PRIMARY KEY (Session_ID, Equipment_ID),  
    FOREIGN KEY (Session_ID) REFERENCES Test_Session(Session_ID),  
    FOREIGN KEY (Equipment_ID) REFERENCES Equipment(Equipment_ID)  
);
```